

Project Design – EduGenie-AI

Google Gemini Powered AI Learning Assistant

Page 1 – Design Overview and Objectives

The Project Design phase converts the requirements of EduGenie-AI into a clear technical and user-oriented structure. The design explains how the learner interacts with the system, how requests are processed, how Google Gemini generates AI responses, and how the different modules work together.

Design Objectives

- Create a simple and student-friendly learning assistant interface.
- Design separate modules for question answering, explanations, quizzes, summaries, and learning recommendations.
- Use a FastAPI backend to manage API requests and application logic.
- Integrate Google Gemini through the Google GenAI Python SDK.
- Keep the system modular so individual features can be improved independently.
- Provide clear responses suitable for college-level learners.

Overall Design Concept

EduGenie-AI follows a client–server architecture. The student interacts with the web interface. The frontend sends a request to the FastAPI backend. The backend identifies the requested learning operation, calls the appropriate module, and returns the generated result to the interface.

Layer	Main Responsibility
User Interface	Collect learner input and display results
FastAPI Backend	Handle routes, validation and response flow
Learning Modules	Perform Q&A, explanation, quiz, summary and recommendation tasks
Gemini AI Layer	Generate natural-language educational content

Page 2 – System Architecture Design

The system architecture defines the major components of EduGenie-AI and the communication between them. The design separates presentation, application logic, AI interaction, and configuration so that the project remains easier to test and maintain.

Architecture Components

- Frontend: HTML, CSS and JavaScript provide the learner-facing web page.
- FastAPI Application: `main.py` contains the web routes and connects the frontend with the learning modules.
- Question Answering Module: `qna.py` processes student questions using Gemini.
- Explanation Module: `explanation_module.py` generates understandable topic explanations.
- Quiz Module: `quiz_module.py` creates quiz-oriented learning content.
- Summary Module: `summary_module.py` produces concise summaries from provided content.
- Learning Path Module: `learning_path.py` handles learning recommendations.
- Environment Configuration: `.env` stores the Gemini API key and model configuration without exposing credentials in source control.

Request and Response Flow

Step	Component	Activity
1	Student	Enters a question or selects a learning feature
2	Frontend	Sends an HTTP request to the required FastAPI endpoint
3	FastAPI	Receives and validates the request
4	Learning Module	Builds the appropriate educational prompt
5	Gemini	Generates the AI response
6	Backend	Returns the result to the frontend
7	Frontend	Displays the learning content to the student

The modular architecture also allows future improvements such as user accounts, progress tracking, a database, authentication, or additional AI learning tools without redesigning the entire application.

Page 3 – Module and Interface Design

EduGenie-AI is designed around learning activities that students commonly need. Each activity is represented as a separate functional area, while the user interface provides a consistent way to submit input and view results.

Core Module Design

Module	Input	Output
Q&A	Student question	Clear AI-generated answer
Explanation	Topic or concept	Step-by-step explanation
Quiz	Topic / subject	Quiz questions and learning content
Summary	Long text or notes	Concise summary
Learning Path	Topic / learning need	Recommended learning sequence

User Interface Design

- A simple landing page introduces EduGenie-AI and its learning capabilities.
- Input fields allow students to enter questions, topics, or text.
- Feature controls allow the student to select Q&A, explanation, quiz, summary, or recommendations.
- A result area presents the generated content in a readable format.
- The interface should avoid unnecessary complexity so students can focus on learning.
- JavaScript handles asynchronous requests so the page can communicate with FastAPI without a complete page reload.

Usability Considerations

The design emphasizes readability, clear labels, predictable interactions, and understandable AI responses. Educational content should be presented with headings, lists, examples, and step-by-step explanations wherever appropriate. The interface should also provide useful feedback when an input is missing or a backend request fails.

Page 4 – AI Integration and Data Flow Design

The AI design defines how EduGenie-AI communicates with Google Gemini while keeping application configuration separate from source code. The Google GenAI Python SDK is used to send prompts to the configured Gemini model.

AI Processing Design

- The application loads GEMINI_API_KEY from the environment configuration.
- The selected Gemini model is read from GEMINI_MODEL.
- Each learning module prepares a task-specific prompt.
- The prompt includes the student's request and instructions for educational clarity.
- The Gemini response is returned to the FastAPI layer.
- The frontend displays the generated result to the learner.

Prompt Design Principles

- Use clear role instructions so Gemini responds as a learning assistant.
- Request simple language suitable for college students.
- Specify the expected task, such as explaining, summarizing, answering, or generating quiz content.
- Avoid unnecessary information and keep responses focused on the learner's request.
- Use structured output instructions when a feature requires a particular format.

Data Flow and Security Design

Student input is sent to the backend and used to construct the relevant AI request. The Gemini API key is not placed in frontend JavaScript or committed to Git. The .env file is excluded using .gitignore. This design reduces the risk of accidentally publishing the secret key.

Error Handling

- Validate empty or invalid input before calling Gemini.
- Return an understandable error when the AI service cannot be reached.
- Avoid exposing secret keys or internal credentials in error messages.
- Keep the application responsive when an external AI request fails.

Page 5 – Design Validation and Future Scalability

The final design is evaluated against the project requirements to ensure that the proposed architecture can support the intended learning features. The design is intentionally modular so that new functionality can be introduced without rewriting the existing application.

Design Validation Checklist

- The frontend can communicate with FastAPI endpoints.
- Each major learning feature has a separate module or clear responsibility.
- The backend can connect to Google Gemini using the configured model.
- API credentials are kept outside the source code.
- Responses can be displayed to students in a readable form.
- The design supports basic error handling and future testing.

Future Design Enhancements

- User authentication and student profiles.
- Database support for storing learning history and quiz results.
- Personalized learning progress dashboards.
- Difficulty-level selection for explanations and quizzes.
- Voice-based question input and response features.
- Multilingual learning support.
- Teacher or administrator dashboards.
- Analytics for identifying frequently requested topics.

Conclusion

The Project Design phase provides the technical blueprint for EduGenie-AI. The proposed architecture connects a simple web interface, FastAPI backend, modular learning components, and Google Gemini AI into one learning assistant. The separation of responsibilities improves maintainability, testing, security, and future scalability. This design forms the foundation for the subsequent Project Planning and Project Development phases.